

Total No. of Questions : 8]

P-7546

SEAT No. :

[Total No. Of Pages : 2

[6180]-54

**T.E. (Computer Engg)
WEB TECHNOLOGY**

(2019 Pattern) (Semester - II) (310252)

Time : 2½ Hours]

[Max. Marks : 70

Instructions to the candidates :

- 1) Answer Q. 1 or Q. 2, Q. 3 or Q. 4, Q. 5 or Q. 6, Q. 7 or Q. 8
- 2) Neat diagrams must be drawn wherever necessary
- 3) Figures to the right indicate full marks
- 4) Assume suitable data, if necessary

Q1) a) Explain the Servlet architecture with diagram and explain servlet lifecycle. **[9]**

b) Explain the concept of DTDs with example. Differentiate DTD Vs XML Schema (Min 4). **[9]**

OR

Q2) a) Explain the Servlet and MySQL database connectivity with example code to display data from employee(emp_id, emp_name, emp_dept) table. **[9]**

b) Write note on: **[9]**

i) AJAX

ii) XML

Q3) a) Explain the concept of JSP with syntax and sample example. Explain the analogy of JSP and Servlets. **[9]**

b) Explain the concept of WSDL and SOAP. **[8]**

OR

P.T.O.

PDF BY VL



Telegram Channel

[Click here to join](#)



Instagram Page

[Click here to join](#)



WhatsApp Channel

[Click here to join](#)



WHY JOIN ?

- 🏆 Trusted by 80,000+ Engineering Students
- 🔔 Stay Updated with latest university news
- 🚀 Make your engineering journey easy with us

Best Regards, **Vishwajeet Londhe**

Q4) a) Explain JSP with support for Model View Controller. **[9]**

b) Explain the concept of struts with architecture, actions, interceptors and exception handling. **[8]**

Q5) a) Explain different types of arrays in PHP with example **[9]**

b) Write note on: **[9]**

i) ASP.NET

ii) NodeJS

OR

Q6) a) Write php code to connect MYSQL database to display records from the table **[9]**

b) Explain the concepts of WAP, WML and .NET framework. **[9]**

Q7) a) Explain scalar types, operations and pattern matching in Ruby. **[9]**

b) Explain documents requests and processing forms in Rails. **[8]**

OR

Q8) a) Explain the concept of classes and arrays in Ruby. **[9]**

b) Explain concepts of Rails with AJAX and EJB. **[8]**



Solutions:

Q1 (a) Explain the Servlet architecture with diagram and explain servlet lifecycle.[9]

Ans:

Servlet architecture defines how a servlet interacts with the **client**, **web server**, and **container**.

Servlet lifecycle describes the different stages a servlet goes through from **creation** → **execution** → **destruction**.

Servlet Architecture

1. Client (Browser)

- User sends request using URL (HTTP request).

2. Web Server

- Receives client request.
- Forwards request to **Servlet Container**.

3. Servlet Container (Tomcat)

Responsible for:

- Loading servlet class
- Creating servlet instance
- Managing servlet lifecycle
- Calling init(), service(), destroy() methods
- Managing threads and security

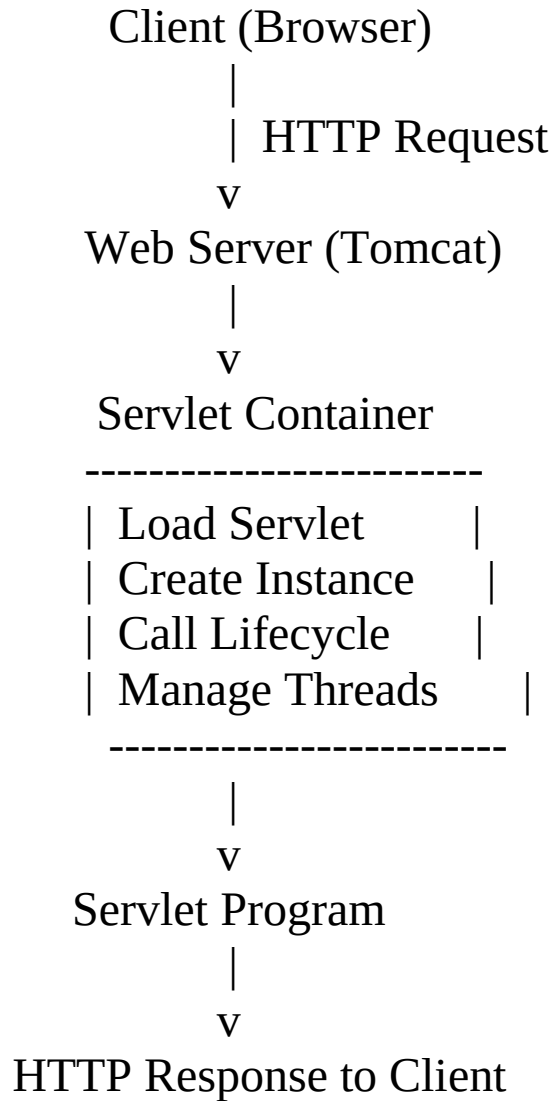
4. Servlet Program

- Java class that processes request and generates dynamic response.

5. Response Back to Client

- Servlet sends an HTML/text output.
- Server forwards it back to client browser.

Servlet Architecture Diagram



Servlet Life Cycle

Servlet lifecycle has **3 main methods** + loading & instantiation.

1. Loading the Servlet

- Performed by the servlet container.
- Happens when the first request arrives or at startup (load-on-startup).

2. Creating Object

- Only **one servlet object** is created.
- Used for all requests (multi-threaded).

3. init() method

- Called **once** in servlet life.
- Used for initialization (DB connection, reading configs).

```
public void init() {  
    // initialization code  
}
```

4. service() method

- Called **for every request**.
- Dispatches request to doGet() or doPost().

```
public void service() { }
```

5. destroy() method

- Called before servlet is removed from memory.
- Used for cleanup (closing DB connections, releasing resources).

```
public void destroy() { }
```

Q1 (b) Explain the concept of DTDs with example. Differentiate DTD vs XML Schema (min 4 points).[9]

Ans:

DTD (Document Type Definition) defines the **structure**, **rules**, and **allowed tags** of an XML document.

It ensures the XML is **valid**.

1. DTD – Explanation (Point-to-Point)

1. Defines structure

- Specifies allowed elements, attributes, child elements.

2. Validates XML

- Checks whether XML follows correct formatting rules.

3. Two types of DTD

- **Internal DTD** (inside XML file)
- **External DTD** (separate .dtd file)

2. DTD Example

Internal DTD Example

```
<?xml version="1.0"?>
<!DOCTYPE student [
  <!ELEMENT student (name, roll)>
  <!ELEMENT name (#PCDATA)>
  <!ELEMENT roll (#PCDATA)>
]>
<student>
  <name>Vijay</name>
  <roll>42</roll>
</student>
```

Explanation

- <student> element must contain <name> and <roll>
- #PCDATA means parsed character data (text)

3. Difference Between DTD and XML Schema (Minimum 4 points)

DTD	XML Schema
1. Written in its own syntax (not XML).	1. Written completely in XML format.
2. Supports only basic data types like #PCDATA.	2. Supports rich datatypes (string, int, date, boolean).
3. Cannot define namespaces.	3. Fully supports XML namespaces.
4. Less powerful and less flexible.	4. More powerful, modern, and widely used.
5. Limited validation abilities.	5. Strong validation rules available.
6. Doesn't support inheritance.	6. Supports element inheritance & reusability.
7. Difficult to extend.	7. Easily extendable and modular.
8. Not suitable for large applications.	8. Used in real-world enterprise applications.

OR

Q2) a) Servlet–MySQL Database Connectivity with Example (9 marks)

Ans:

Servlet can connect to a MySQL database using **JDBC (Java Database Connectivity)**.
The basic steps are:

Steps for Connecting Servlet to MySQL

1. Load the JDBC Driver

`Class.forName("com.mysql.cj.jdbc.Driver");`

2. Create Connection

Use `DriverManager.getConnection()` with database URL, username, password.

3. Create Statement / PreparedStatement

Write SQL query.

4. Execute Query

Use `executeQuery()` to get data.

5. Process ResultSet

Display rows inside HTML using `PrintWriter`.

6. Close connection.

Example Servlet Code: Display Data From Employee Table

Table:

employee(emp_id, emp_name, emp_dept)

```
import java.io.*;
import java.sql.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class DisplayEmployee extends HttpServlet {
    public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        try {
            // 1. Load driver
            Class.forName("com.mysql.cj.jdbc.Driver");

            // 2. Create connection
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/companydb", "root", "password");

            // 3. Query
            PreparedStatement ps = con.prepareStatement("SELECT * FROM employee");

            // 4. Execute
            ResultSet rs = ps.executeQuery();

            // 5. Display data
            out.println("<h2>Employee Details</h2>");
            out.println("<table border='1'>");
            out.println("<tr><th>ID</th><th>Name</th><th>Department</th></tr>");

            while (rs.next()) {
                out.println("<tr>");
                out.println("<td>" + rs.getInt("emp_id") + "</td>");
                out.println("<td>" + rs.getString("emp_name") + "</td>");
                out.println("<td>" + rs.getString("emp_dept") + "</td>");
                out.println("</tr>");
            }
            out.println("</table>");

            // 6. Close
            con.close();

        } catch (Exception e) {
            out.println("Error: " + e);
        }
    }
}
```

Q2) b) Write Note On: AJAX & XML (9 marks)

Ans:

i) AJAX (Asynchronous JavaScript and XML)

AJAX is a technique used to create **fast, dynamic, and responsive** web applications.

Meaning

- AJAX allows web pages to update **data in background** without refreshing the entire page.
- It uses JavaScript to send/receive data from the server.

How AJAX Works (Simple Flow)

1. User performs an action (example: click button).
2. JavaScript creates an **XMLHttpRequest** object.
3. Request goes to server **without reloading page**.
4. Server sends response.
5. JavaScript updates only required part of the webpage.

Advantages

- Faster updates
- No full page reload
- Better user experience
- Low bandwidth use

Real-Life Examples

- Google Search suggestions
- Instagram likes update instantly
- Live cricket score

ii) XML (Extensible Markup Language)

XML is used to **store and transport data** in a structured format.

Features

- Self-descriptive tags
- Platform independent
- Hierarchical structure
- Supports data sharing between systems

Structure of XML

1. **XML Declaration**
2. `<?xml version="1.0" encoding="UTF-8"?>`
3. **Root element** (Only one root)
4. **Child elements**
5. **Attributes and nested tags**

Example

```
<employee>  
  <emp_id>101</emp_id>  
  <emp_name>Rohan</emp_name>  
  <emp_dept>IT</emp_dept>  
</employee>
```

Uses of XML

- Data exchange (Web services)
- Configuration files
- RSS feeds
- SOAP messages

Q3 (a) Explain the concept of JSP with syntax and sample example. Explain the analogy of JSP and Servlets. (9 marks)

Ans:

JSP is a server-side technology used to create **dynamic web pages** using a combination of HTML + Java code.

It allows developers to write most of the page in HTML and insert Java code only where required.

JSP = “*HTML mixed with Java (server-side)*”

It is easier to write than servlets because HTML is written normally without Java print statements.

2. JSP Syntax

(i) JSP Directive

Used to give instructions to the container.

```
<%@ page import="java.util.*" %>
```

(ii) Declaration Tag

Used to declare variables or methods.

```
<%! int num = 10; %>
```

(iii) Scriptlet Tag

Used to write Java code inside JSP.

```
<% int a = 5; int b = 10; int c = a + b; %>
```

(iv) Expression Tag

Used to directly print output.

Result: `<%= c %>`

3. Sample JSP Example

Example: sum.jsp

```
<%@ page language="java" %>
```

```
<html>
```

```
<body>
```

```
<%
```

```
    int a = 10;
```

```
    int b = 20;
```

```
    int sum = a + b;
```

```
%>
```

```
<h2>Sum is: <%= sum %></h2>
```

```
</body>
```

```
</html>
```

4. Analogy (Relationship) Between JSP and Servlets

Key Idea:

JSP and Servlets both generate dynamic web content.
Difference is only in coding style.

Important Points (Exam-ready):

1. **JSP is internally converted into a Servlet.**
 - JSP → converted into Java servlet file → compiled → executed.
2. **JSP is HTML-centric, Servlets are Java-centric.**
 - JSP focuses on UI (View).
 - Servlet focuses on processing (Controller).
3. In MVC architecture:
 - JSP = View
 - Servlet = Controller
4. **Servlets use Java code to write HTML** using `out.println()`, while **JSP uses HTML and embeds Java** using `<% %>`.
5. **JSP is good for presentation**, Servlets are good for business logic.

Q3 (b) Explain the concept of WSDL and SOAP. (8 marks)

Ans:

1. Concept of WSDL (Web Services Description Language)

Concept in Brief

WSDL is an **XML document** that describes everything about a web service — what it does, where it is located, and how to call it.

WSDL = “*Web service contract*”

Key Features of WSDL

1. Written in **XML format**.
2. Specifies the **operations (methods)** offered by the web service.
3. Defines **input and output data types**.
4. Specifies **URL** where the service is available.
5. Used by clients to call the service.

WSDL Structure (Important)

A WSDL file contains:

1. **types** – data types used
2. **message** – input/output parameters
3. **portType** – list of operations
4. **binding** – protocol (SOAP/HTTP)
5. **service** – endpoint (actual URL)

Simple WSDL Example (Snippet)

```
<definitions>
  <message name="getWeatherRequest">
    <part name="city" type="xsd:string"/>
  </message>

  <portType name="WeatherService">
    <operation name="getWeather"/>
  </portType>
</definitions>
```

2. Concept of SOAP (Simple Object Access Protocol)

Concept in Brief

SOAP is a protocol used for sending and receiving messages between a **client and server** in a web service.

It uses **XML format** for communication.

SOAP = “XML messaging protocol”

Features of SOAP

1. Platform independent
2. Uses XML for request/response
3. Works over HTTP, SMTP
4. Very secure
5. Suitable for enterprise applications

SOAP Message Structure

```
<Envelope>  
  <Header>...</Header>  
  <Body>...</Body>  
  <Fault>...</Fault>  
</Envelope>
```

Example SOAP Message

```
<Envelope>  
  <Body>  
    <getWeather>  
      <city>Pune</city>  
    </getWeather>  
  </Body>  
</Envelope>
```

Q4 a) Explain JSP support for Model-View-Controller (MVC).[9]

Ans:

1. MVC Basics

MVC divides application into 3 parts:

- **Model** → Data & business logic
- **View** → UI (JSP)
- **Controller** → Request handling (Servlet)

2. JSP Support for MVC

1. JSP as View

- JSP displays data to the user.
- No business logic should be inside JSP.
- Gets data from servlet using request.setAttribute().

2. Servlet as Controller

- Accepts request
- Processes input
- Calls Model (JavaBeans / classes)
- Stores data in request scope
- Forwards to JSP

3. JavaBeans as Model

- JavaBeans contain business logic, database logic, validations.
- Provide getter/setter methods for data.

3. MVC Flow Diagram

Client → Controller (Servlet) → Model (JavaBean) → Controller → View (JSP) → Client

4. Example Workflow

- User submits login form → Servlet
- Servlet validates using JavaBean
- Result forwarded to JSP
- JSP displays “Login Success/Failed”

5. Advantages

- Clear separation of logic & UI
- Reusable components
- Easy maintenance
- Scalable web apps

Q4 b) Explain Struts with architecture, actions, interceptors & exception handling.[8]

Ans:

1. What is Struts?

- Struts is a **Java MVC framework** used to build enterprise web applications.
- Follows MVC design pattern.

2. Struts Architecture Components

1. ActionServlet (Controller)

- Main controller
- Receives all client requests
- Reads config file (struts.xml)

2. Action class

- Contains business logic
- Each request maps to an Action class
- Returns “success”, “error”, etc.

3. JSP (View)

- Forms and displays output
- Uses Struts tags for UI

4. Model

- Java classes, services, database logic

5. Configuration file (struts.xml)

- Maps URL → Action class → Result JSP

3. Flow of Struts MVC

Client → ActionServlet → Action Class → Business Logic → returns result → JSP View

4. Interceptors

- Pre-processing and post-processing tasks
- Example tasks:
 - Validation
 - Login check
 - File upload
 - Logging

Every request passes through interceptors before reaching Action class.

5. Exception Handling

- Defined in struts.xml
- On exception → mapped JSP page is shown
- Helps avoid server crash and shows friendly error page.

6. Example Code Snippet

struts.xml

```
<action name="login" class="com.app.LoginAction">
  <result name="success">welcome.jsp</result>
  <result name="error">error.jsp</result>
</action>
```

Action Class

```
public class LoginAction extends ActionSupport {
  private String user, pass;

  public String execute() {
    if(user.equals("admin") && pass.equals("123")) {
      return "success";
    }
    return "error";
  }
}
```

Q5) a) Explain different types of arrays in PHP with example. [9]

Ans:

An array in PHP stores multiple values in one variable. Arrays can be indexed, associative, or multidimensional.

Types of Arrays

1) Indexed Array

- Stores values with **numeric index** (0,1,2...)
- Useful when order matters.
- PHP automatically assigns the index.

Example

```
<?php
$colors = array("Red", "Green", "Blue");
echo $colors[1]; // Output: Green
?>
```

2) Associative Array

- Uses **string keys** instead of numbers.
- Useful for key-value data (like name, age, city).

Example

```
<?php
$student = array(
    "name" => "Vijay",
    "age" => 22,
    "city" => "Pune"
);
echo $student["city"]; // Output: Pune
?>
```

3) Multidimensional Array

- Array containing one or more arrays.
- Useful for table-like data (records of students, products, etc.)

Example

```
<?php
$employees = array(
    array("101", "Rohit", "HR"),
    array("102", "Amit", "IT"),
    array("103", "Sneha", "Finance")
);

echo $employees[1][1]; // Output: Amit
?>
```

4) Mixed Array

- PHP arrays can contain **different data types together**.

Example

```
<?php
$mixed = [10, "Hello", 12.5, true];
?>
```

5) Array with User-defined Keys

- PHP allows custom numeric keys.

Example

```
<?php
$data = array(
    10 => "A",
    20 => "B"
);

echo $data[20]; // Output: B
?>
```

Q5) b) Write note on: ASP.NET & NodeJS. [9]

Ans:

i) ASP.NET

Concept

ASP.NET is a **Microsoft web development framework** used to build dynamic websites and web applications.

Explanation

1. ASP.NET is part of the **.NET Framework**.
2. It allows creation of **server-side web apps** using C#.
3. Supports **Web Forms, MVC, Web API, and Razor Pages**.
4. Provides rich server-side controls like:
 - TextBox
 - Button
 - GridView
5. Supports **code-behind model** (UI and logic separated).
6. Automatically manages:
 - Memory
 - Security
 - Session
 - State management
7. Provides built-in caching for performance.
8. Can integrate with SQL Server, MySQL, and other databases.

ii) NodeJS – Pointwise Note

Concept in Brief

NodeJS is a **JavaScript runtime environment** that executes JavaScript on the **server side**.

Explanation

1. Built on **Google Chrome V8 engine**.
2. Uses **event-driven, non-blocking I/O**, making it fast and scalable.

Best suited for **real-time applications** like chat apps and streaming.

Uses **npm (Node Package Manager)** which has thousands of packages.

Written in JavaScript, making it easy for frontend developers to write backend too.

Supports asynchronous programming using:

- Callbacks
- Promises
- async/await

Common frameworks:

- ExpressJS
- NestJS

OR-Q6) a) Write PHP code to connect MySQL database and display records from a table. [9]

Ans:

PHP connects to MySQL using MySQLi or PDO. After connecting, a query is executed and results are displayed.

PHP Code (MySQLi – Procedural)

```
<?php
// Step 1: Connect to database
$conn = mysqli_connect("localhost", "root", "", "college");

// Check connection
if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}

// Step 2: Execute query
$sql = "SELECT emp_id, emp_name, emp_dept FROM employee";
$result = mysqli_query($conn, $sql);

// Step 3: Display records
if (mysqli_num_rows($result) > 0) {
    while($row = mysqli_fetch_assoc($result)) {
        echo "ID: ".$row["emp_id"]." - Name: ".$row["emp_name"]." - Dept:
        ".$row["emp_dept"]."<br>";
    }
} else {
    echo "No records found";
}

// Step 4: Close connection
mysqli_close($conn);
?>
```

Q6) b) Explain the concepts of WAP, WML, and .NET Framework. [9]

Ans:

1) WAP (Wireless Application Protocol)

Concept in Brief

WAP is a protocol that allows **mobile devices** to access the internet.

Explanation

1. Developed for **small-screen, low-bandwidth** devices.
2. Provides a standard way for mobiles to access web content.
3. Uses **WML** (Wireless Markup Language).
4. Works over **GSM, CDMA, GPRS** networks.
5. WAP architecture includes:
 - Mobile device
 - WAP Gateway
 - Internet server

2) WML (Wireless Markup Language)

Concept in Brief

WML is a markup language used to create **mobile web pages** for WAP.

Explanation

1. Similar to HTML but specifically designed for mobile screens.
2. Pages are divided into **cards** inside a **deck**.
3. WML tags include:
 - <wml>
 - <card>
 - <p>
4. Supports user input, navigation, and simple scripting.
5. Content is interpreted by a **WAP micro-browser**.

3) .NET Framework

Concept in Brief

A Microsoft platform for building Windows and web applications.

Explanation

1. Includes **CLR (Common Language Runtime)**.
2. Uses **BCL (Base Class Library)** for ready-made classes.
3. Supports multiple languages like:
 - C#
 - VB.NET
 - F#
4. Provides:
 - Memory management
 - JIT compilation
 - Garbage collection
5. ASP.NET is built on .NET Framework.
6. Supports secure and scalable enterprise-level applications.

Q7 a) Explain scalar types, operations and pattern matching in Ruby.[9]

Ans:

1) Scalar Types in Ruby

Scalar types store single/simple values.

i) Integer

- Stores whole numbers

a = 10

ii) Float

- Stores decimal values

x = 5.5

iii) String

- Sequence of characters

name = "Ruby"

iv) Boolean

- Stores **true/false**

flag = true

v) Nil

- Represents "no value"

data = nil

2) Operations on Scalar Types

a) Arithmetic Operations

a + b

a - b

a * b

a / b

a % b

b) Comparison Operations

a == b

a != b

a > b

a < b

c) Logical Operations

&& || !

d) String Operations

"Hello " + "Ruby"

"Hi" * 3 # "HiHiHi"

"Name: #{name}"

3) Pattern Matching in Ruby

Ruby uses **regular expressions (RegEx)** with the =~ or match operator.

i) Using =~ Operator

```
if "Hello Ruby" =~ /Ruby/  
  puts "Pattern found"  
end
```

ii) Using match Method

```
result = "2025".match(/\d+/)  
puts result[0] # 2025
```

iii) Checking string format

```
email = "test@mail.com"  
if email =~ /\A[\w.]+\@\w+\.\w+\z/  
  puts "Valid email"  
end
```

iv) Substitution

```
puts "color".gsub(/o/, 'u') # "culur"
```

Q7 b) Explain Document Requests and Processing Forms in Rails.[8]

Ans:

1) Document Requests in Rails

Rails follows the **Request–Response cycle**.

a) Steps in Document Request

1. **Browser sends request** (GET/POST).
2. **Router** decides which controller should handle the request.
3. **Controller** processes request.
4. **Controller calls Model** (if database interaction is needed).
5. **Data is passed to View**.
6. **View generates HTML** response.
7. **Browser displays output**.

b) Example Flow

Request: /students

- Router → students#index
- Controller → fetches students
- View → renders index.html.erb

2) Processing Forms in Rails

a) Creating a Form (ERB)

```
<%= form_with url: "/students", method: :post do |form| %>
  <%= form.text_field :name %>
  <%= form.submit "Save" %>
<% end %>
```

b) Controller Method

```
def create
  @student = Student.new(student_params)
  @student.save
end
```

c) Strong Parameters

```
def student_params
  params.require(:student).permit(:name)
end
```

d) Advantages of Rails Form Processing

- Automatic parameter handling
- Built-in validation
- CSRF protection
- Easy integration with database via ActiveRecord

OR-Q8 a) Explain the concept of classes and arrays in Ruby.[9]

Ans:

1) Classes in Ruby

A class defines **objects**, their **data**, and **methods**.

a) Creating a Class

```
class Student
  def initialize(name, age)
    @name = name
    @age = age
  end

  def display
    puts "#{@name} - #{@age}"
  end
end
```

b) Creating Object

```
s = Student.new("Vijay", 20)
s.display
```

c) Features

- Everything is an object
- Instance variables (@var)
- Constructor method (initialize)
- Public/private methods

2) Arrays in Ruby

a) Creating Array

```
arr = [10, 20, "Ruby", 3.5]
```

b) Accessing

```
arr[0] # 10
```

c) Adding Elements

```
arr << 99
```

d) Removing Elements

```
arr.delete(20)
```

e) Iterating

```
arr.each do |x|
  puts x
end
```

f) Useful Methods

- length
- sort
- reverse
- push
- pop

Q8 b) Explain concepts of Rails with AJAX and EJB.[8]

Ans:

1) Rails with AJAX

a) Meaning

AJAX = Asynchronous JavaScript and XML

Used to update part of a webpage **without full reload**.

b) Rails AJAX Features

- remote: true forms and links
- .js.erb files for JavaScript response

c) Example

Form:

```
<%= form_with url: "/students", remote: true do |f| %>
  <%= f.text_field :name %>
  <%= f.submit "Add" %>
<% end %>
```

create.js.erb:

```
alert("Student added!");
```

d) Advantages

- Faster page interaction
- Less server load
- Better user experience

2) EJB (Enterprise Java Beans)

a) Meaning

EJB is a **server-side component** used to develop **large, secure, scalable enterprise applications** in Java.

b) Types of EJB

1. Session Beans

- Stateless
- Stateful
- Singleton

2. Message Driven Beans

- Handle asynchronous messages

c) Features

- Transaction management
- Security
- Remote access
- Reusable components

d) Applications

- Banking
- E-commerce
- Enterprise systems

PDF BY VL



Telegram Channel

[Click here to join](#)



Instagram Page

[Click here to join](#)



WhatsApp Channel

[Click here to join](#)



WHY JOIN ?

- 🏆 Trusted by 80,000+ Engineering Students
- 🔔 Stay Updated with latest university news
- 🚀 Make your engineering journey easy with us

Best Regards, **Vishwajeet Londhe**